

SYSTEM ARCHITECTURE SPECIFICATION

Audio Classification Architecture

01
Ingestion

02
DSP

03
Feature QA

04
Learning

05
Validation

06
Quantize

07
Runtime

Contents

Seven phases, one continuous pipeline — plus the full page-by-page studio reference.

01 Executive Overview & System Architecture

The seven-phase pipeline at a glance

02 Phase-by-Phase Implementation Details

Phases 1–7: ingestion through real-time runtime

03 Studio UI Walkthrough & Reference

All 8 workflow pages, field-by-field

04 Project Timeline & Effort Estimation

Per-phase hours, deliverables, and total project duration

SECTION 01

Executive Overview & System Architecture

This specification details the complete, phase-by-phase design for a custom, end-to-end **Audio Classification Architecture**. It matches the capabilities of Edge Impulse's sound recognition and audio pipeline, covering everything from data acquisition to on-device embedded inference.

PHASE 1 Data Ingestion & Dataset Curation

- 16 kHz PCM audio ingestion
- Auto-slicing (1000 ms window, 300 ms stride)
- Speaker / session train-test split
- Audio augmentation (noise, shift, gain)

**PHASE 2 DSP Preprocessing Engine**

- MFE (Mel-filterbank energy) · MFCC (speech) · raw STFT spectrogram
- Identical mathematical representation between Python & embedded C/C++

**PHASE 3 Feature Explorer & Data Quality**

- Dimensionality reduction (PCA / UMAP)
- 2D / 3D cluster visualization
- Bad-data, silence & outlier detection before training

**PHASE 4 Learning Block & Model Architectures**

- Choice A — train from scratch (lightweight 1D / 2D CNN)
- Choice B — transfer learning (pre-trained audio backbones)
- Choice C — anomaly detection (K-means / GMM clustering)
- Choice D — dual-head model (classification + anomaly detection)

**PHASE 5 Validation, Profiling & Active Learning**

- Confusion matrix & F1-score
- Hardware profiler (latency & peak SRAM)
- Independent test dataset evaluation
- Active learning sample-moving loop

**PHASE 6 Quantization & Compilation — "EON" Engine**

- Full INT8 post-training quantization (PTQ) with calibration dataset
- Memory arena optimization (RAM reuse)

- Export to embedded C array (model.h)



PHASE 7 Continuous Real-Time Runtime Engine

- Circular DMA audio ring buffer
- Slice-based rolling DSP updates (e.g. 250 ms)
- Debouncing & confidence thresholding
- Microcontroller (C++) & PC (Python) targets

SECTION 02

Phase-by-Phase Implementation Details

Phase 1 — Data Ingestion, Slicing & Augmentation

1.1 Audio Standardization

All incoming audio is converted into a standard TinyML audio representation:

- **Sampling rate (f_s):** 16,000 Hz (16 kHz)
- **Channels:** 1 (mono)
- **Bit depth:** 16-bit PCM signed integers (normalized to $[-1.0, 1.0]$ floats during DSP)

1.2 The Window Slicing Algorithm

Continuous recordings (e.g. 60-second audio files) are sliced using a sliding window:

- **Window size:** 1000 ms (16,000 samples)
- **Window increase (stride):** 300 ms (4,800 samples)

$$\text{Overlap} = (1000 - 300) / 1000 = 70\%$$

A single 10-second recording produces 31 distinct training samples instead of only 10, drastically improving model generalization.

1.3 Split Strategy (Preventing Data Leakage)

Never split sliced windows randomly. If windows from the same 60-second recording appear in both training and test sets, the model overfits by memorizing background acoustics. Split **by source file or speaker ID**: 80% of original recordings go to data/training/, and 20% go to data/testing/.

1.4 Data Augmentation Suite

During training, raw waveforms undergo real-time augmentation:

1. **Background noise mixing** — inject random background silence, traffic, or white noise at dynamic signal-to-noise ratios (SNR: 5–20 dB).
2. **Time-shifting** — roll audio horizontally by ± 100 –200 ms.
3. **Gain / volume jitter** — multiply waveform by a random scalar $\alpha \in [0.8, 1.2]$.
4. **SpecAugment** — mask random blocks of frequencies or time frames during training to prevent co-adaptation of features.

Phase 2 — Digital Signal Processing (DSP) Engine

Raw audio (16,000 numbers per second) contains excessive high-frequency noise and temporal variance. The DSP block converts raw 1D time-domain data into a compact 2D time-frequency matrix.

2.1 The MFE (Mel-Filterbank Energy) Pipeline

Recommended for **non-vocal sound recognition** (faucet, alarms, machine sounds):

1. **Pre-emphasis** — boost high frequencies:

$$y[t] = x[t] - \alpha \cdot x[t-1], \quad \alpha \approx 0.97$$

2. Framing & windowing:

- Frame length: 32 ms (512 samples at 16 kHz)
- Frame stride: 20 ms (320 samples)
- Window function — Hanning window: $w[n] = 0.5 - 0.5 \cdot \cos(2\pi n / (N-1))$

3. FFT (Fast Fourier Transform) — compute 512-point FFT power spectrum:

$$P[k] = (1/N) \cdot |X[k]|^2$$

4. Mel filterbank — apply $M = 40$ triangular overlapping bandpass filters spaced along the perceptual Mel scale:

$$m = 2595 \cdot \log_{10}(1 + f/700)$$

5. Log energy — calculate the logarithm of the filterbank energies:

$$E_m = \log(\sum P[k] \cdot H_m[k] + \epsilon)$$

6. Output matrix:

≈ 49 time frames $\times$ 40 Mel frequency bands = **1,960 features** per window
Compared to 16,000 raw samples, this represents an 87.7% reduction in input dimensionality.

2.2 DSP Block Selection Guide

Block	Best suited for
MFE	Environmental sounds, running water, glass breaks, gunshots, industrial machinery.
MFCC	Human voice commands, wake words, keyword spotting (adds a DCT step to capture the vocal-tract formant envelope).
Spectrogram	Continuous vibration sensors, motor frequency resonance.

Phase 3 — Feature Explorer & Quality Validation

Before training a model, the extracted features are audited to verify that distinct acoustic classes separate into clean clusters.

Dimensionality Reduction

Run **PCA** (Principal Component Analysis) or **UMAP** (Uniform Manifold Approximation and Projection) on the 1,960-dimensional feature vectors, projecting them into 2D or 3D coordinates.

Visual Quality Checks

- **Cluster separation** — distinct classes should form separated clusters.
- **Mislabeled-data detection** — a sample landing inside the wrong cluster is auto-flagged for review.
- **Silence filtering** — reject windows where total RMS energy falls below the noise threshold.

Phase 4 — Learning Block Selection & the Model Library

Selecting how your model trains is a **two-step selection process**:

4.1 — Impulse Design: choose learning block type →

4.2 — Classifier page: "Choose a different model" opens the Model Library

Step 4.1 — Learning Block Type Selection (in "Create Impulse")

Clicking + **Add a learning block** selects the high-level category:

1. **Classification (Neural Network)** — learns to categorize inputs into your labels (faucet, noise, etc.)
2. **Anomaly Detection (K-Means)** — unsupervised clustering to detect unexpected sounds without labels
3. **Regression** — continuous numeric estimation

You can add both **Classification** and **Anomaly Detection** simultaneously for dual-mode inference.

Step 4.2 — The Model Preset Library ("Model Zoo")

Inside the **Classifier** page, clicking **Choose a different model** opens the model architecture catalog:

Model preset	Mode	Peak RAM	Flash ROM	Target hardware & use case
Tiny Conv2D	Train from scratch	~41.9 KB	~45.8 KB	Small microcontrollers (ESP32, STM32 Cortex-M4, Arduino Nano BLE). Ultra-fast, fits under 50 KB RAM.
MobileNetV1 0.1	Transfer learning	~45.0 KB	~122.7 KB	Speech & wake words. Pre-trained acoustic weights fine-tuned on custom samples — high accuracy with low data.
MobileNetV2 0.35	Deep transfer learning	~228.0 KB	~5,707 KB	High accuracy / edge gateways (ESP32 with PSRAM, Cortex-M7, Raspberry Pi, Jetson, PC).
Syntiant NDP10x	ASIC accelerator	Ultra-low	Ultra-low	Specialized ultra-low-power neural hardware (Syntiant microwatt chips).

Expert View	Bring your own model	Custom	Custom	Raw Keras / PyTorch / scikit-learn Python code editor for data scientists.
--------------------	----------------------	--------	--------	--

Phase 5 — Validation, Hardware Profiling & Active Learning

5.1 Validation Metrics

A 20% validation split is held out from the training set, producing: **categorical accuracy, confusion matrix, precision, recall, and F1-score.**

5.2 Hardware Profiler

The build tool outputs realistic embedded hardware estimates:

- **Inference latency estimate** — layer-by-layer FLOPs mapped to ARM Cortex-M4 @ 80 MHz clock cycles:

$$\text{Estimated Latency} \approx (\text{MACCs} \times 2) / \text{Clock Frequency (Hz)}$$

- **Peak SRAM profiler** — simulates TensorFlow Lite Micro's double-buffered tensor arena to verify memory safety.
- **ROM / flash footprint** — total size of quantized model weights and bias tensors.

5.3 Independent Model Testing & Active Learning Loop

1. Run the model against the completely isolated data/testing/ dataset.
2. Generate an itemized test report showing accuracy per individual audio clip.
3. **Active learning mechanism** — if an audio clip yields low confidence (e.g. 50–65%), a one-command workflow migrates that sample to the training set and triggers retraining.

Phase 6 — Model Quantization & Compilation ("EON" Engine)

6.1 Full Integer INT8 Post-Training Quantization (PTQ)

Floating-point models (float32) are too slow and bulky for microcontrollers without a hardware FPU. All weights, biases, and intermediate activations are converted from float32 to int8:

$$q = \text{round}(r / S) + Z$$

Where S is the scale factor and Z is the zero-point offset.

Representative calibration dataset: 100 real MFE windows are fed through the converter to calibrate dynamic ranges.

6.2 C Header Export (model_data.h)

The quantized .tflite model is converted into an embedded C byte-array:

```
#ifndef MODEL_DATA_H_
#define MODEL_DATA_H_

alignas(16) const unsigned char g_model[] = {
    0x1c, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33, ...
};
```

```
const unsigned int g_model_len = 18456; // 18.4 KB

#endif // MODEL_DATA_H_
```

Phase 7 — Real-Time Continuous Runtime Engine

`run_classifier_continuous` — on real devices, sounds happen unpredictably. Slicing audio sequentially causes high latency, so the continuous engine uses sliding ring buffers.

7.1 Circular Ring Buffer & Rolling DSP

- **Audio DMA buffer** — the microcontroller's I2S DMA continuously writes audio samples into a circular RAM buffer holding the last 1,000 ms (16,000 samples).
- **Slice updates** — every 250 ms (4,000 new audio samples):
 1. Oldest 250 ms of DSP features are shifted out.
 2. MFE is computed only for the newest 250 ms slice.
 3. INT8 model inference runs on the updated 1,000 ms feature buffer.

Latency is reduced from **1,000 ms down to only 250 ms** by only recomputing the newest slice on every update.

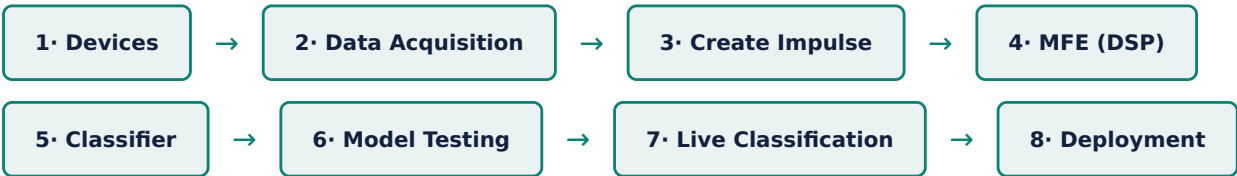
7.2 Post-Processing, Debouncing & Peak Filtering

- **Confidence threshold** — require $P(\text{target}) \geq 0.80$ to trigger a positive detection.
- **Debounce / suppression window** — once triggered, enforce a 1.5-second cooldown timer to prevent multiple triggers for the same continuous sound.

SECTION 03

Complete Page-by-Page Studio UI Walkthrough & Reference

The complete visual reference for all 8 studio pages in their exact chronological order — what the user sees, what fields exist, and how each step operates.



Page 1 — Devices (Hardware Connection)

HARDWARE

Purpose: connect a development board, smartphone, or local PC microphone so the system can stream real audio.

Devices

+ Connect a New Device

Device Name	Type	Sensors	Status
Laptop Microphone	PC / WebUSB	Audio (16 kHz)	Connected
Arduino Nano 33 BLE Sense	Serial Daemon (COM)	Audio, Accel	Connected
ESP32-S3 Eye	Remote Management	Camera, Mic	Offline

Want to record from your phone? **Show QR Code** — scan to open the mobile recorder in your browser.

User actions: click a device to pair. A green **Connected** status enables sampling.

Page 2 — Data Acquisition (Recording & Labelling)

DATASET

Purpose: record audio, assign labels (faucet, noise), and manage training (80%) and testing (20%) datasets.

Data Acquisition

Training Data (80%)

RECORD NEW DATA

Device

Laptop Microphone

Sensor

Built-in Mic (16000Hz)

Sample length

10000 ms

Label

faucet

Start Sampling

Or drag & drop .WAV files — auto-split train/test (80/20) enabled.

COLLECTED SAMPLES — TOTAL 10m 00s

File	Label	Len
faucet.01.wav	faucet	10.0s
faucet.02.wav	faucet	10.0s
noise.01.wav	noise	10.0s
noise.02.wav	noise	10.0s

Preview: faucet.01.wav

Key fields: the **Label** field defines classes dynamically. **Start Sampling** records standard 16 kHz audio.

Page 3 — Create Impulse (Pipeline Design)

DESIGN

Purpose: set window slicing, connect the DSP feature extraction block, and attach the classification model.

Create Impulse

Save Impulse

TIME SERIES DATA

Window size1000 ms

Window stride300 ms

Frequency16000 Hz

Zero-pad✓ enabled

PROCESSING BLOCK (DSP)

Audio (MFE)

Mel-filterbank energy

InputsTime series

Outputs1,960 features

LEARNING BLOCK (MODEL)

Classifier

Neural Network (Keras)

InputsMFE features

Outputs2 classes

User actions: set the 1000 ms window and 300 ms stride (70% overlap multiplier), then click **Save Impulse** — this activates the MFE and Classifier sidebar tabs.

Page 4 — MFE (DSP Feature Extraction & Visual Explorer)

DSP

Purpose: convert raw audio waveforms into 2D spectrogram matrices and visually audit cluster quality with the 3D Feature Explorer.

MFE (Mel-Filterbank Energy)

Generate Features

DSP PARAMETERS

Frame length0.032 s

Frame stride0.020 s

Filter number40

FFT length512

Low frequency300 Hz

High frequency8000 Hz

Noise floor−52 dB

FEATURE EXPLORER — 3D CLUSTERING

faucet cluster

noise cluster

— well separated

Inspect dotfaucet_12.wav

CoordinatesX 1.42, Y −0.85, Z 3.12

Generation status100% complete

Key visual: the 3D Feature Explorer clusters samples by acoustic similarity using UMAP/PCA. Outlier dots reveal mislabeled or silent recordings before training.

Page 5 — Classifier (Model Selection & Training)

TRAINING

Purpose: open the Model Zoo to select between Tiny Conv2D or MobileNetV1, train the network, and review on-device RAM/latency profiling.

Classifier (Neural Network)

Start Training

MODEL & TRAINING SETTINGS

ArchitectureTiny Conv2D

Input49×40×1 (MFE)

Epochs30

Learning rate0.001

Validation split20%

Training log: Epoch 30/30 — Loss 0.04 — Acc 98.4%

VALIDATION & HARDWARE PROFILER

Accuracy98.4%

	faucet	noise	F1
faucet	98.1%	1.9%	0.98
noise	1.2%	98.8%	0.99

Inferencing time142 ms

Peak RAM18.4 KB

Flash (ROM)42.1 KB

User actions: click **Choose a different model** to select architecture, adjust epochs, click **Start Training**, and review the confusion matrix and peak RAM.

Page 6 — Model Testing (Rigorous Test Suite & Active Learning)

EVALUATION

Purpose: evaluate the trained model against unseen test data and move difficult samples to the training set.

Model Testing

Overall Accuracy: 89.2%

Sample	Expected	Predicted	Confidence	Result
test_faucet_01.wav	faucet	faucet	99%	100%
test_noise_04.wav	noise	noise	97%	100%
test_faucet_08.wav	faucet	noise (wrong)	58%	0%

Active learning action on the failed row → **Move to training set** (retrain to fix this weakness).

Active learning loop: if a test recording produces low confidence or fails, use the row menu → **Move to training set** to improve the model.

Page 7 — Live Classification (Microphone Test Stream)

LIVE TEST

Purpose: test the model with live streaming microphone audio before flashing it to a physical board.

Live Classification

Start Sampling

Device

Laptop Microphone

Sample length

5000 ms

Time	0.0s	1.0s	2.0s (tap ON)	3.0s	4.0s	5.0s
faucet	2%	3%	98%	97%	95%	4%
noise	98%	97%	2%	3%	5%	96%

Timeline split into 41 rolling windows — visual bar highlights the **FAUCET DETECTED** span.

User actions: click **Start Sampling**, trigger the sound in real life, and verify continuous predictions over the timeline.

Page 8 — Deployment (Quantization & Export)

EXPORT

Purpose: package the trained model into standalone C++ code, an Arduino ZIP, or a C byte-array header (model_data.h).

Deployment

Build & Download

1. DEPLOYMENT TARGET

★ Selected	C++ Library	Includes DSP + model weights in standalone C++ source
Available	Arduino Library (.ZIP)	Ready to install in Arduino IDE for ESP32 / Nano 33 BLE
Available	WebAssembly (WASM)	Run directly in browser JavaScript web apps
Available	C Header (model_data.h)	Raw INT8 array for bare metal
Available	Linux / Raspberry Pi (.eim)	Pre-compiled Linux binary
Available	Flashable Firmware Binary	Direct USB flash to board

2. QUANTIZATION & OPTIMIZATION

● Quantized (INT8) — selected

RAM ~18.4 KB (75% savings)

Latency ~142 ms (3× faster)

○ Unoptimized (Float32)

RAM ~73.6 KB

Latency ~420 ms

SECTION 04

Project Timeline & Effort Estimation

Realistic, task-validated effort estimate for one engineer building this entire pipeline — from raw audio ingestion through the 8-page studio dashboard — working full-time (8-hour days).

Phase	What Happens In This Phase	Est. Hours	Est. Days
Phase 1 Data Ingestion	Build the 16 kHz mono ingestion pipeline, code the 1000 ms / 300 ms sliding-window slicer, set up speaker/file-level train-test split, and implement the 4-part augmentation suite (noise mixing, time-shift, gain jitter, SpecAugment).	4 hrs	0.5 day
Phase 2 DSP Engine	Implement MFE, MFCC and spectrogram extractors from first principles (pre-emphasis, framing, FFT, Mel filterbank, log-energy), then verify numerical parity between the Python reference and the embedded C/C++ version.	16 hrs	2.0 days
Phase 3 Feature Explorer	Build the PCA/UMAP dimensionality-reduction pipeline, the 2D/3D cluster visualizer, and the automated silence, outlier and mislabeled-sample detection checks.	8 hrs	1.0 day
Phase 4 Learning Block & Models	Build the model factory (scratch Tiny Conv2D, MobileNetV1/V2 transfer learning, K-means/GMM anomaly detection, dual-head classifier), write the training loop, and run architecture comparisons for accuracy vs. footprint.	16 hrs	2.0 days
Phase 5 Validation & Profiling	Build the confusion-matrix/F1 validator, the Cortex-M4 latency and peak-SRAM estimator, the independent test-set evaluator, and the active-learning "move to training set" retrain loop.	12 hrs	1.5 days
Phase 6 Quantization	Implement INT8 post-training quantization with a representative calibration dataset, validate the accuracy drop stays acceptable, and build the C-header exporter for the quantized model.	12 hrs	1.5 days
Phase 7 Real-Time Runtime	Build the circular DMA ring buffer, the 250 ms rolling-DSP update logic, confidence-threshold debouncing, and both the embedded C++ and PC Python live-inference runners.	16 hrs	2.0 days
Subtotal	Core 7-phase pipeline (data → DSP → model → quantization → runtime), excluding the dashboard UI.	84 hrs	10.5 days

Studio UI 8-Page Dashboard	Build the Devices, Data Acquisition, Create Impulse, MFE, Classifier, Model Testing, Live Classification and Deployment pages — front-end UI only, wired to mocked/placeholder data rather than the live backend.	8 hrs	1.0 day
Integration & Testing	Connect every phase into one working end-to-end pipeline, test with real recorded audio on target hardware, fix edge cases, and polish the UI.	16 hrs	2.0 days
TOTAL PROJECT	Full pipeline (7 phases) + 8-page studio UI + integration & testing.	108 hrs	13.5 days

108 TOTAL HOURS	13.5 WORKING DAYS	~2.7 WEEKS (FULL-TIME)
--------------------	----------------------	---------------------------